

ENG4200 Coursework 2 – Group Report

Group ID: Telco-CW2-C

Group Members (Name + ID):

- Ayush Bhandge (3148137B)
- Prathamesh Deepak Bagal (3154253B)
- Karthik Mandigiri (3155271M)
- Nathan Sidi Bakari (3071212S)
- Zefu Wang (3136524W)
- Jiangyue Yao Zhao (3084311Z)

Dataset: Telco Customer Churn

1. Objective & Dataset (5–6 sentences)

The goal of this coursework is to design and critically evaluate supervised learning pipelines for predicting customer churn on the Telco Customer Churn dataset, formulated as a binary classification problem. Following the general machine-learning workflow (problem understanding, data preparation, model selection, evaluation, and deployment), we emphasise experiment design and justification rather than raw benchmark accuracy. The input space contains a mixture of numerical features (e.g. tenure, both monthly and total charges) and high-cardinality categorical variables (e.g. contract type, payment method), with moderate class imbalance (approximately 26.5% churners). This structure motivates careful preprocessing and an evaluation strategy that explicitly addresses imbalance and overfitting. In line with the coursework brief, we compare several models from the supervised learning part of the course (logistic regression, decision trees, support vector machines, random forests and gradient boosting) using a common experimental protocol based on stratified train–test splitting and cross-validation. Our analysis focuses on three axes: predictive performance (F1-score and ROC–AUC), computational efficiency (training and prediction time), and robustness to overfitting, leading to a cost-aware choice of a primary deployed model and an interpretable baseline.

2. Pipeline Designs

All pipelines share the same high-level structure, consistent with the standard estimator–transformer API of scikit-learn:

- Data cleaning:** string trimming and coercion of `TotalCharges` to numeric; collapsing “No internet service” and “No phone service” into a single “No” level, to avoid spurious sparsity and simplify the feature space.
- Preprocessing:** a *ColumnTransformer* with median imputation and standardisation for numerical features, and most-frequent imputation plus one-hot encoding for categorical features, using `drop="if_binary"` to avoid redundant dummy variables.
- Classifier:** one of the course models, with hyperparameters tuned in the individual notebooks by grid search over theoretically motivated ranges.

We implemented five pipelines, which can be grouped into three conceptual families (linear, non-linear single-tree, and ensemble/kernel methods). Table 1 summarises their key design choices and complexity controls.

Figure 1 can be used in the presentation as a single comparative flow diagram illustrating the common data-to-prediction pathway.

[Pipeline diagram: *Raw data* → Cleaning → ColumnTransformer (numeric & categorical branches) → Classifier (LR / DT / SVM / RF / GB)]

Figure 1: Placeholder for the shared preprocessing + model selection pipeline.

3. Comparative Analysis

We follow the model-evaluation rules from the course: a single stratified 80/20 train-test split, with all hyperparameter tuning and cross-validation performed exclusively on the training set. Within the training set, we use 5-fold stratified cross-validation with shuffled splits to obtain reliable estimates of generalisation performance and overfitting gaps. Because churners form the minority class and the business cost of false negatives is high, we focus on the F1-score as the primary metric, complemented by accuracy and ROC-AUC. F1 directly reflects the harmonic mean of precision and recall, thus penalising models that either miss too many churners (low recall) or raise too many false alarms (low precision), while ROC-AUC captures ranking quality in a threshold-independent way.

Table 2 reports cross-validated performance on the training set, while Table 3 summarises the final benchmark on the held-out test set together with training and prediction times.

To satisfy the coursework requirement of analysing accuracy, efficiency and overfitting in a single view, we also produced the following summary figures:

- **[Insert figure: F1 vs. training time scatter plot]**. Each point corresponds to one model; the efficient frontier in this plane comprises only the Decision Tree and Random Forest, as they are not dominated in both F1 and training cost.
- **[Insert figure: ROC-AUC vs. training time scatter plot]**. This highlights that Gradient Boosting achieves the highest ROC-AUC but with markedly lower F1, indicating that its ranking ability is high but its default 0.5 threshold under-utilises recall on the minority class.
- **[Insert figure: bar chart of cross-validated overfitting gaps]**. This visualises the

fact that the single Decision Tree has the largest train–validation F1 gap, consistent with its high variance, whereas Logistic Regression displays almost no gap, in line with its high bias and strong regularisation.

Overall, Random Forest dominates in F1 on both cross-validation and test sets, with competitive ROC–AUC and modest computational cost. Logistic Regression provides a well-calibrated linear baseline; SVM and Gradient Boosting offer strong ROC–AUC but either incur a substantial training cost (SVM) or sacrifice F1 (Gradient Boosting); and the shallow Decision Tree is fast but clearly less stable, as predicted by the bias–variance analysis of high-variance single trees versus ensembles.

Figure 2: Placeholder for the table/figure comparing **accuracy, efficiency, and overfitting**.

4. Chosen Pipeline

We selected the **Random Forest pipeline** as the primary model for deployment, with the Decision Tree pipeline retained as an interpretable reference. The choice is grounded both in empirical evidence and in the theoretical properties discussed in the course:

- **Best trade-off between F1 and cost.** On the held-out test set the Random Forest achieves the highest F1-score (0.630) while remaining on the efficient frontier: its training time (≈ 0.27 s) is only marginally higher than that of the simple Decision Tree, yet its F1 improves by roughly 0.9 percentage points and its ROC–AUC is significantly higher (0.845 vs. 0.816). From a cost-sensitive perspective where false negatives are expensive, this gain in F1 is worth the negligible additional training cost.
- **Robust generalisation via bagging.** Bagging theory shows that averaging M decorrelated models reduces variance approximately by a factor $1/M$ (or, more generally, to

$\sigma^2(\rho + (1 - \rho)/M)$ when predictions have correlation ρ), without increasing bias when base learners share the same bias. Our Random Forest, built from 200 trees with bootstrap sampling and random feature selection, empirically exhibits lower train–test gaps than the single tree, confirming the expected variance reduction.

- **Non-linear decision boundaries without manual feature engineering.** Decision trees and their ensembles partition the feature space into axis-aligned hyper-rectangles and thus can approximate complex, heterogeneous decision surfaces that a linear model cannot capture. In the churn context, this allows the model to capture interactions such as “short tenure *and* month-to-month contract *and* fibre optic” without explicit cross-features.
- **Diagnostics support the choice.** The Random Forest confusion matrix shows a favourable balance between recall and precision on churners; its ROC and precision–recall curves present a smooth, high-performing trade-off across thresholds; and its learning curve displays converging training and validation F1, indicating that adding more data would yield diminishing returns and that the model is not grossly underfitting.
- **Interpretability via feature importances.** Although less transparent than the Decision Tree, the Random Forest still admits global interpretation through impurity-based feature importances, which rank contract type, tenure, total and monthly charges, fibre-optic internet and electronic cheque as key drivers (consistent with domain intuition and hence increasing trust in the model).

The Decision Tree pipeline remains valuable as a human-readable baseline: its shallow depth makes it suitable for communicating rules to business stakeholders, even if its variance and lower ROC–AUC render it less attractive as the primary automated decision-maker.

5. Team Reflection

Each team member briefly summarises their contribution and a key insight gained from the project.

- **A. Bhandge:** [1–2 sentences on contributions, e.g. implementation of the Random Forest and Gradient Boosting pipelines, and insight gained about bias–variance trade-offs and ensemble methods.]
- **P. D. Bagal:** [1–2 sentences on contributions, e.g. development of the logistic regression and SVM pipelines, and learning about the effect of regularisation and kernel parameters on margin and generalisation.]
- **K. Mandigiri:** [1–2 sentences on contributions, e.g. data cleaning, EDA and construction of the unified Jupyter notebook, and insight into the importance of a disciplined evaluation protocol (train/validation/test separation, metric choice).]
- **N. Sidi Bakari:** [1–2 sentences on contributions, e.g. implementation of the Random Forest and Gradient Boosting pipelines, and insight gained about bias–variance trade-offs and ensemble methods.]
- **Z. Wang:** [1–2 sentences on contributions, e.g. development of the logistic regression and SVM pipelines, and learning about the effect of regularisation and kernel parameters on margin and generalisation.]
- **J. Y. Zhao:** [1–2 sentences on contributions, e.g. data cleaning, EDA and construction of the unified Jupyter notebook, and insight into the importance of a disciplined evaluation protocol (train/validation/test separation, metric choice).]

Appendix (optional, max 1 page)

The following additional figures, not all included in the main body due to space constraints, further support our analysis and are available in the accompanying notebook:

- **[EDA plots: tenure, MonthlyCharges, contract type].** Histograms and bar plots illustrating key marginal distributions and the class imbalance.

- **[F1 vs. training time and ROC–AUC vs. training time scatter plots]**. Used to construct the efficient frontier and identify dominated models.
- **[Learning curve for the Random Forest]**. Showing training and validation F1 as a function of training-set size, supporting the claim of moderate variance and limited underfitting.
- **[Random Forest ROC and precision–recall curves]**. Visualising the trade-off between recall and precision across thresholds.
- **[Top-15 feature importance bar chart]**. Summarising the most influential features in the Random Forest ensemble.

These figures, together with the reproducible Jupyter notebook, ensure that the experimental process is transparent and that all claims in the report can be independently verified.

Pipeline		Base model	Key hyperparameters / complexity control	Theoretical motivation
Logistic Regression		ℓ_2 -regularised logistic regression	$C = 0.01$, penalty = ℓ_2 , solver SAGA, class_weight = balanced	Linear decision boundary with probabilistic outputs; the ℓ_2 penalty controls $\ w\ _2$ and hence variance, implementing regularisation that trades a small increase in bias for a substantial reduction in overfitting risk.
Decision Tree		CART decision tree classifier	Max depth = 3, min samples split = 2, min samples leaf = 1, class_weight = balanced	Non-linear, axis-aligned splits chosen greedily to maximise impurity reduction (Gini). Restricting depth acts as structural regularisation, limiting the size of the tree and therefore its variance.
SVM (RBF kernel)		Soft-margin SVM with RBF kernel	$C = 0.1$, $\gamma = \text{scale}$, class_weight = balanced, probability = True	Maximises the geometric margin in feature space while allowing slack variables. The RBF kernel implements a flexible non-linear mapping; the pair (C, γ) controls capacity: small C and moderate γ encourage a wider margin and better generalisation.
Random Forest		Random Forest classifier (bagging of trees)	$n_estimators = 200$, max depth = 12, min samples split = 20, min samples leaf = 1, max features = log2, class_weight = balanced	Bagging aggregates many de-correlated trees, reducing variance by averaging. Bootstrap sampling and random feature subsampling reduce correlation between trees, approximating the classical $1/M$ variance reduction of independent estimators while retaining low bias.
Gradient Boosting		GradientBoosting classifier (additive trees)	$n_estimators = 200$, learning rate = 0.05, max depth = 2, subsample = 0.8	Boosting builds an additive model of shallow trees fitted to the negative gradients of the loss. Low depth and shrinkage (learning rate) regularise the ensemble, implementing a bias-variance trade-off controlled by the learning rate and number of stages.

Table 1: Summary of pipelines: all share the same cleaning and preprocessing, differing only in the final estimator and its complexity control.

Model	F1 (CV mean)	F1 (CV std)	ROC-AUC (CV mean)	Overfitting gap (F1)
Random Forest	0.640	0.013	0.845	≈ 0.04
Logistic Regression	0.632	0.011	0.843	≈ 0.01
SVM (RBF)	0.623	0.013	0.840	≈ 0.02
Decision Tree	0.619	0.015	0.816	≈ 0.05
Gradient Boosting	0.593	0.016	0.850	≈ 0.03

Table 2: Cross-validated performance (5-fold CV on the training set). Overfitting gap denotes $\text{mean}(\text{train F1}) - \text{mean}(\text{validation F1})$.

Model	Train [s]	Predict [s]	Accuracy	F1	ROC-AUC
Random Forest	0.27	0.03	0.767	0.630	0.845
Decision Tree	0.04	0.01	0.746	0.621	0.816
Logistic Regression	0.06	0.01	0.743	0.619	0.838
SVM (RBF)	3.99	0.27	0.791	0.614	0.835
Gradient Boosting	0.69	0.01	0.799	0.571	0.846

Table 3: Benchmark on the held-out test set (threshold 0.5 for all probability-based models). Training and prediction times are wall-clock seconds on the same machine.